ECEN 248 - Lab Report

Lab Number: 5

Lab Title: Introduction to Logic Simulation and Verilog

Section Number: 509

Student's Name: Saira Khan

Date: 10/19/2022

TA: Sri Kodi

Objectives:

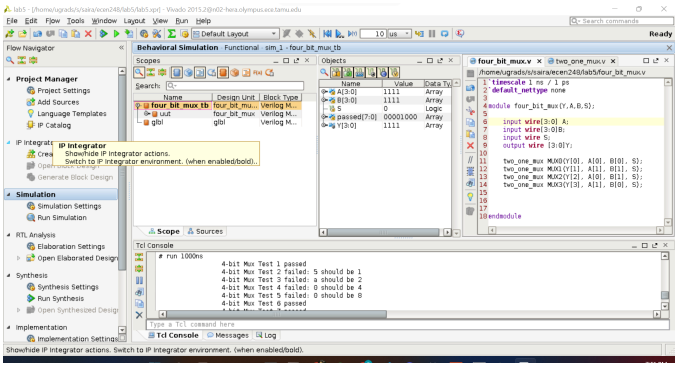
The purpose of this lab is to learn about Verilog and how to use the program Vivado to build and test circuits instead of testing them with breadboards. Through this lab, a 2:1 MUX, full adder, and ALU unit will be implemented through Vivado using previous labs to help. An understanding of Verilog commands and the interface is gained through the lab as well.

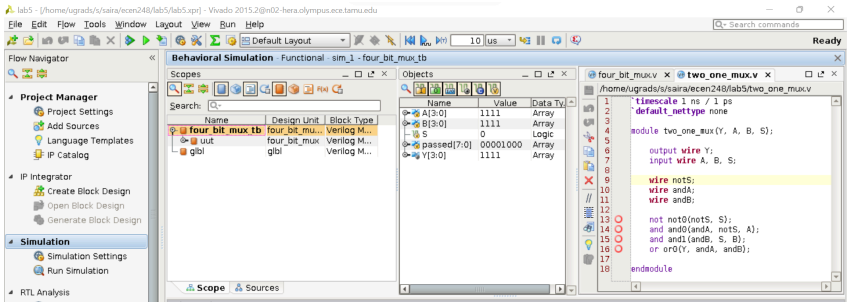
Design:

To complete this lab, Vivadio was first launched using a terminal window and a new design project was created. A source file was then created for the 2:1 MUX and the associated Verilog code was typed into the text file. After this, the 2:1 MUX file was added, along with a test bench, to the Vivadio project by adding it to the sources. A simulation was then run using the test bench, and if any errors were found they were fixed and another simulation was created. This simulation then created a waveform. For the second experiment, modules were designed to build a 4-bit ALU. First, a file for a 4-bit 2:1 multiplexer was created using a starter code that was modified to fit the requirements of the multiplexer. For this, the 2:1 MUX file was called to help complete the build. Both the file and the corresponding test bench were added to the sources of the project, and the test bench was simulated, creating another waveform. Next, a source file was created for the full adder. For this file, the keyword assign was introduced to help describe how data flows from one wire to the next using logic and arithmetic operators. A starter code was provided and modified accordingly. Once again, the file and its test bench were added to the sources and a simulation was run to create a waveform. Then a source file was created for the addition/subtraction unit, implementing the full adder file. The starter code was modified, the file and test bench were added as sources, and another simulation created another waveform. For experiment three, a source file was created for a 4-bit ALU unit. Using the starter code and the previous codes created, the file was created and added as a source along with the test bench. The test bench was then simulated to create the last waveform.

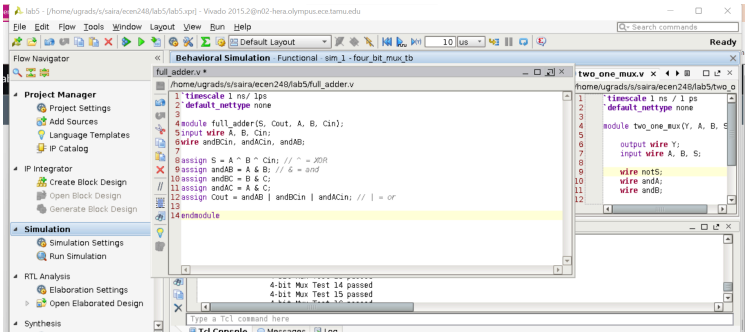
Results:

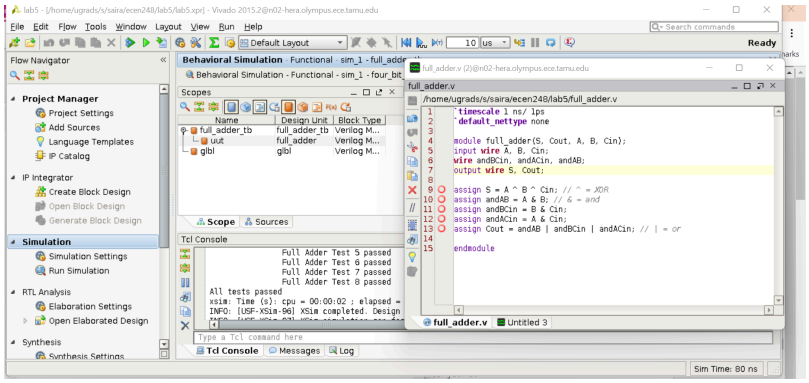
The Vivadio program, though confusing at first, did work a lot like a breadboard in a way. Once the terminology and interface were explored, the logic worked the same as a breadboard did. The circuits were being implemented in the same way but without having to manually put them together.



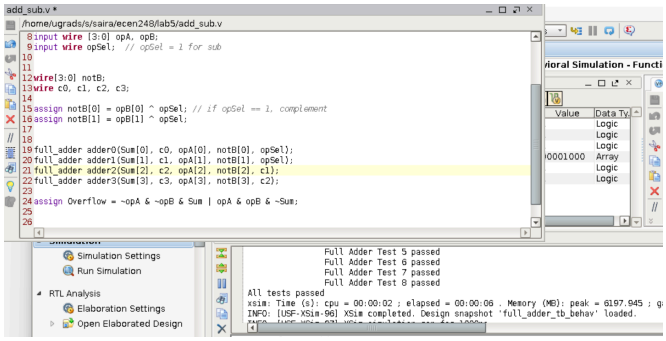


Code for 4-bit 2:1 MUX

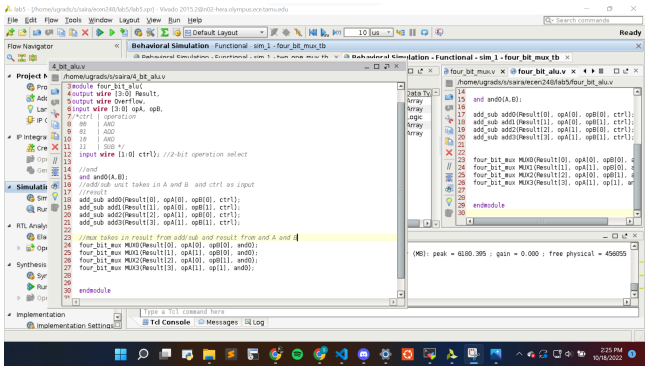




Code for Full Adder



Add/Sub



4-bit ALU

Conclusion:

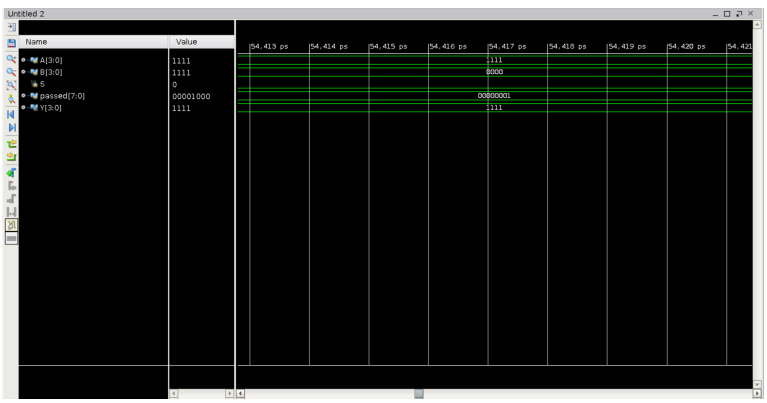
From this lab, a 4-bit ALU circuit was created through a program called Vivadio. Vivadio helped build and test the circuit by writing code simulating the circuit/device. Logic and arithmetic operators were used to help and a test bench was run using the code to create a waveform. From the lab, a better understanding of Vivadio was gained as well as how to create/design circuits digitally instead of using breadboards, which can get too small for certain designs.

Post-lab Deliverables:

The 2:1 test bench for the MUX is first checking the output of the circuit made and comparing it against a known input. A variable is incremented to keep track of the number of tasks passed as well and outputs the results. The code intializes four variable inputs and one variable output. It then sets the variables to 0 before changing the values for certain test cases. Not all the possible test cases are being tested because there would be a lot more and these ones sufficiently conclude whether or not the code works well.

Breadboarding and circuit simulation have similarities and differences. With breadboarding, the circuits are manually implemented by using transistors, wires, power sources, resistors, and LEDs. Circuit simulation, on the other hand, deals with implementing circuits using coding. Both techniques need a strong understanding of the circuit being implemented and the logic associated with it. The advantage of breadboarding is that it is more of a manual test, allowing the user to see how the circuit can be physically used. However, breadboarding can only be used with smaller circuits. With circuit simulation, larger circuits can be implemented in an efficient manner. It doesn't rely on physical components, running out of wires, broken transistors, etc. I prefer breadboarding mostly because I'm more familiar with it than with circuit simulation and the terminology associated with Vivadio. HDLs could be preferred over schematics due to their efficiency. Schematics take more time to draw out and can get messy depending on the size of the circuit. While HDLs can lay out how the circuit works in a cleaner manner. Though by describing a circuit through an HDL, some sense of how the circuit works are

lost. With a schematic, it is easy to follow the input to the output and see what gates it passes through, but with HDL, certain terminology and syntax can be harder to decode right away. I again prefer schematics due to the familiarity. Structural abstraction builds a circuit based on truth tables/already-known information. It works when the user is building a component that is commonly used such as a MUX. Dataflow has more to do with what happens between each gate. It's like following a circuit schematic with your finger, but writing out the logic that's going on instead of drawing gates. It can be used for more complicated circuits that need to be broken down.

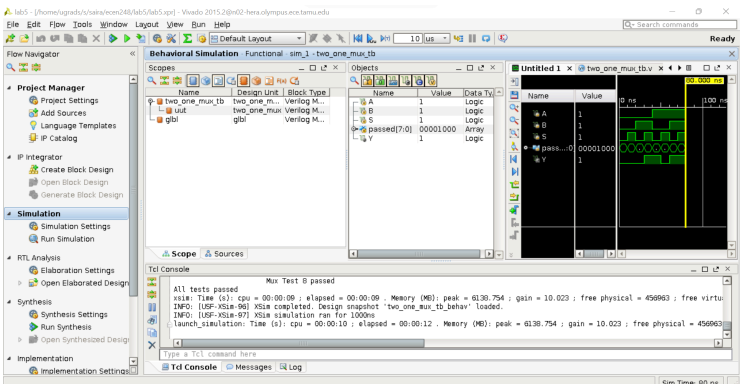


4-bit 2:1 MUX

Full Adder

Untitled 1
Name
Value
opA[3:0]
opB[3:0]
opSel
Result_c...
Overflow...
passed[...
Sum[3:0]
Overflow
1110
1101
1
0100
0
00000000
XX10
1
0 ns
50 ns
100 ns
150 ns
200 ns
250 ns
300 ns
320.000 ns

Add/Sub


File Edit Flow Tools Window Layout View Run Help
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Behavioral Simulation - Functional - sim_1 - two_one_mux_tb
Scopes
Objects
Untitled 1
two_one_mux_tb.v
Tcl Console
Mux Test B passed
All tests passed
Type a Tcl command here
Tcl Console
Messages
Log